Homework 1 – NanoRender

Name: Farida Dabit

ID: 212693345

Course: Computer Graphics

In this assignment I explored the NanoRender framework and implemented several modifications to the framebuffer, user interface, rendering behavior, and application state. The goal was to gain familiarity with low-level rendering concepts, immediate-mode GUI systems, and interactive graphics programming.

Part 1 – Manipulating the Framebuffer

The goal of this task was to modify the framebuffer rendering code and generate a custom 2D visual pattern using mathematical expressions based on the pixel coordinates (x, y).

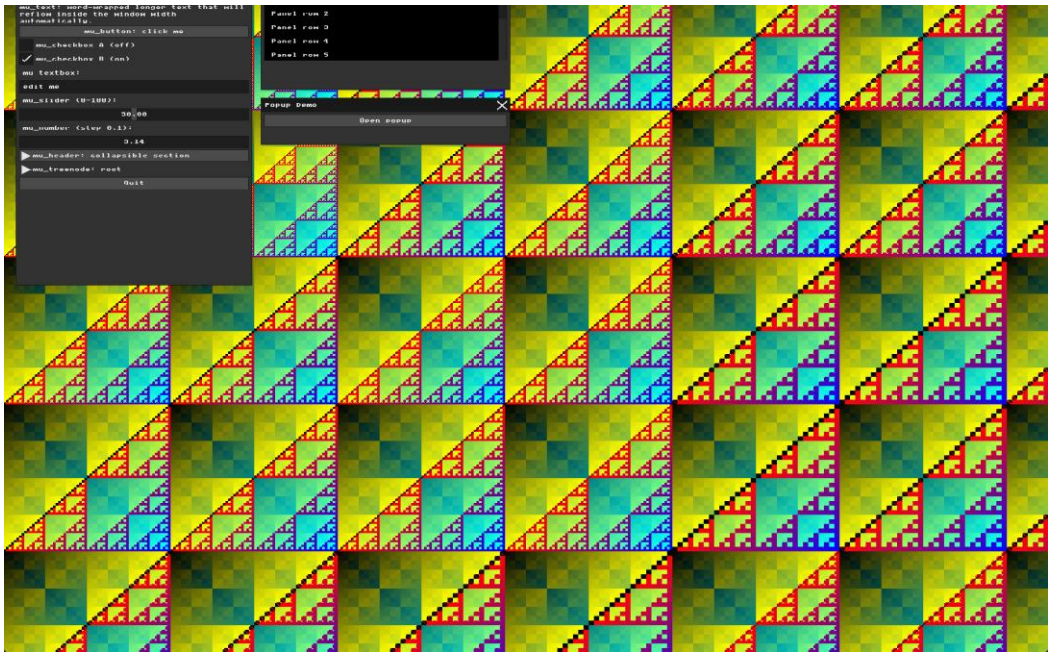
In main.cpp, inside the scene rendering loop, I replaced the original color calculation with new expressions for the red, green, and blue channels:

```
51  + uint8_t r = (x ^ y) % 255;
52  + uint8_t g = (x | y) % 255;
53  + uint8_t b = (x & y) % 255;
54  + g_buffer[i] = MFB_RGB(r, g, b);
```

The color of each pixel is calculated according to its position on the screen. The red channel depends on the multiplication of x and y, the green channel depends on their sum, and the blue channel uses the XOR operation between them.

Result

The resulting image is a colorful two-dimensional pattern that varies across both the horizontal and vertical directions of the screen. Unlike a solid color or a one-dimensional stripe pattern, the generated image changes continuously based on both coordinates and therefore satisfies the assignment requirements.



The modulo operation keeps the RGB values within a valid range. Using multiplication, addition, and XOR creates a rich two-dimensional color pattern across the framebuffer. Since the image depends on both the x and y coordinates, it satisfies the assignment requirements.

Part 2: Immediate Mode UI Declaration

The goal of this task was to add a new interactive widget to the MicroUI interface and practice working with the Immediate Mode programming model.

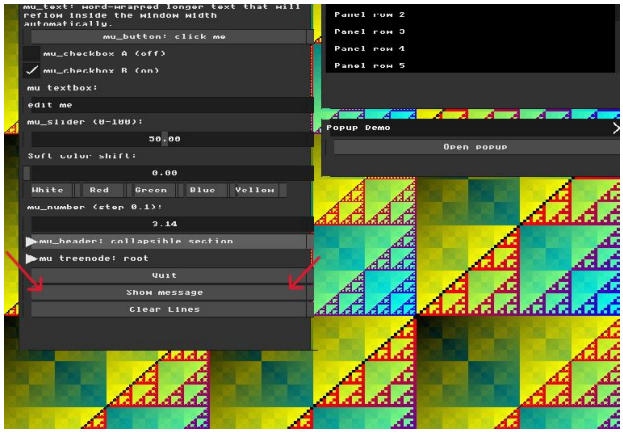
In main.cpp, inside the mu_begin(ctx) block and the Widgets window, I added a new button called "Show message". When the button is pressed, it toggles a state variable and prints a message to the console.The variable show_message is stored outside the widget and keeps track of whether the label should be displayed. Each frame, the

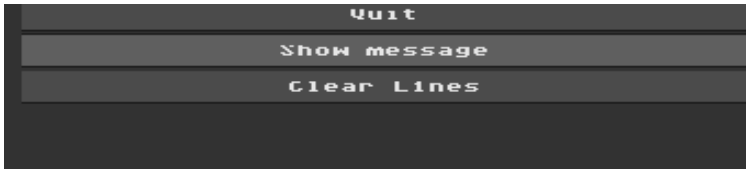
UI is recreated and uses the current value of this variable to decide whether to draw the text.

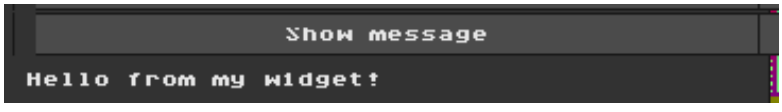
```
+        mu_layout_row(ctx, 1, w1, 0);
+
+ if (mu_button(ctx, "Show message")) {
+      show_message = !show_message;
+      printf("Show message button clicked!\n");
+ }
+
+ if (show_message) {
+      mu_label(ctx, "Hello from my widget!");
+ }
```

Result

A new button was successfully added to the MicroUI window. Clicking the button shows or hides the text "Hello from my widget!" and prints a message to the console. This demonstrates the Immediate Mode approach, where widgets do not store their own state and instead rely on external application variables.







Part 3: The Real-Time Graphics Loop and Input Handling

The goal of this task was to understand how keyboard input is processed through the event loop and how callbacks can be used to modify the application state in real time.

In main.cpp, I modified the character input callback registered with mfb_set_char_input_callback. I added a new state variable called background_mode and used the keyboard key **C** to cycle between different background rendering modes.

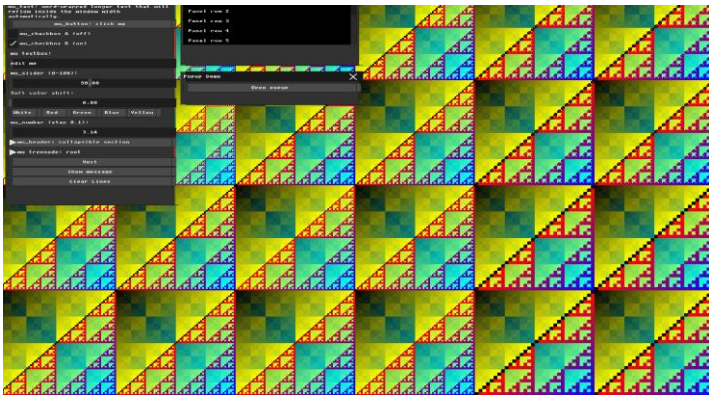
When the user presses **C**, the callback updates the value of background_mode. This value is then used inside the rendering loop to determine which color gradient should be drawn. Because the rendering loop runs continuously, the visual change appears immediately on the screen. The event is consumed by the callback and is not forwarded to the UI text input system.

```
static int background_mode = 0;
```

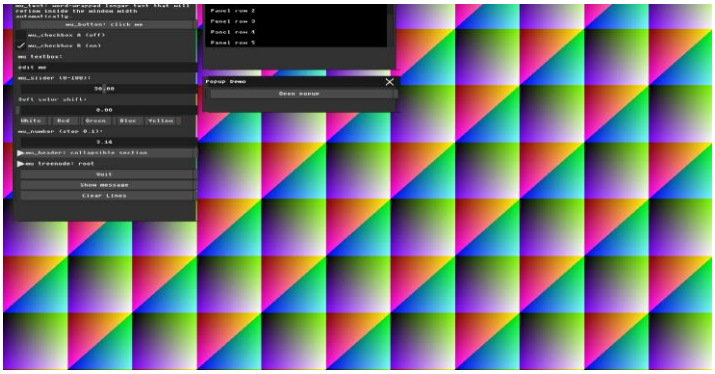
```
// Set up char input callback for textbox input
mfb_set_char_input_callback(
    [](struct mfb_window *w, unsigned int c)
    {
      if (c == 'c' || c == 'C')
      {
        background_mode = (background_mode + 1) % 3;
        return;
      }
      extern void ui_bridge_char_input(struct mfb_window *, unsigned int);
      ui_bridge_char_input(w, c);
    },
    window);
```

Result

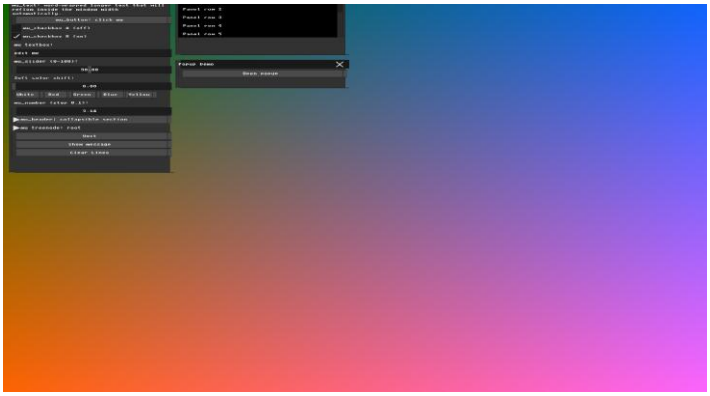
Pressing the **C** key successfully switches between three different background modes in real time. Each mode uses a different color gradient calculation, producing a distinct visual appearance across the framebuffer. This demonstrates the connection between keyboard input, application state, and rendering behavior within the event loop. The task also illustrates how callbacks can be used to create interactive graphics effects that respond instantly to user actions.



Background Mode 0



Background Mode 1

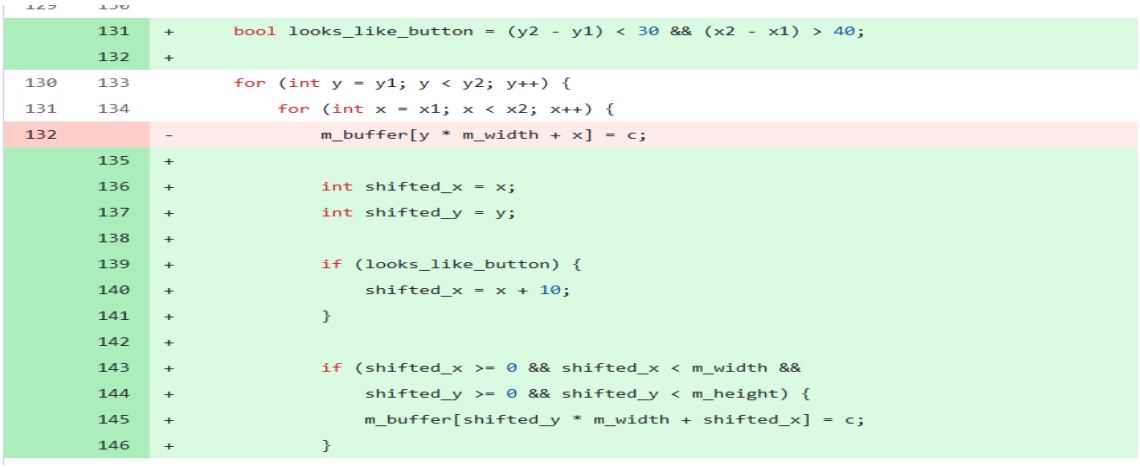


Background Mode 2

Part 4: UI Architecture & The Renderer Bridge

The goal of this task was to understand the separation between the UI logic handled by MicroUI and the visual rendering performed by the renderer. The renderer receives drawing commands from MicroUI and converts them into actual pixels in the framebuffer.

In ui_renderer.cpp, inside the draw_rect() function, I modified the pixel rendering process by applying a small horizontal offset to button rectangles before writing them into m_buffer. The transformation was performed only during rendering and did not modify the original UI layout or input handling logic.



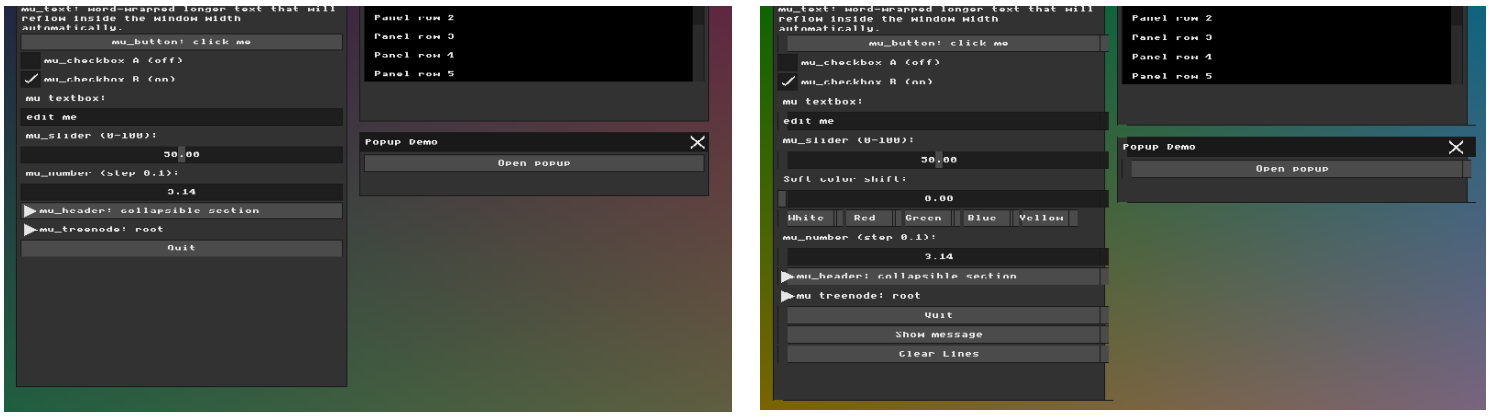
```
+       bool looks_like_button = (y2 - y1) < 30 && (x2 - x1) > 40;
+
        for (int y = y1; y < y2; y++) {
            for (int x = x1; x < x2; x++) {
-               m_buffer[y * m_width + x] = c;
+
+               int shifted_x = x;
+               int shifted_y = y;
+
+               if (looks_like_button) {
+                   shifted_x = x + 10;
+               }
+
+               if (shifted_x >= 0 && shifted_x < m_width &&
+                   shifted_y >= 0 && shifted_y < m_height) {
+                   m_buffer[shifted_y * m_width + shifted_x] = c;
+               }
```

As a result, the visual position of the button background was shifted slightly to the right, while the text and the clickable area remained at their original coordinates.

Result

After recompiling the application, the buttons appeared visually offset from their original position. However, clicking directly on the shifted button did not always activate it. This happened because MicroUI calculates widget positions and mouse interactions independently from the renderer. The renderer only changes how pixels are drawn on the screen and has no effect on the actual UI logic.

To successfully activate the button, the mouse cursor had to be placed at the button's original location rather than on the shifted visual representation. This demonstrates the separation between UI state management and visual rendering in an Immediate Mode GUI system.



Comparison between the original interface and the modified interface after implementing additional UI controls. The updated version includes interactive color controls and extra widgets added during later stages of the assignment.

Part 5: Binding UI to Application State

The goal of this task was to create a new application state variable, bind it to a UI widget, and use it to dynamically affect the rendered image. This demonstrates how Immediate Mode GUI widgets interact with external application data through memory pointers.

In main.cpp, I created a new application state variable:

```
89    static float color_shift = 0.0f;
```

Then I added a new slider widget inside the Widgets window:

```
232           mu_label(ctx, "Soft color shift:");
233           mu_slider(ctx, &color_shift, 0, 100);
```

The slider receives a pointer to color_shift, allowing the widget to directly modify the variable when the user interacts with it.

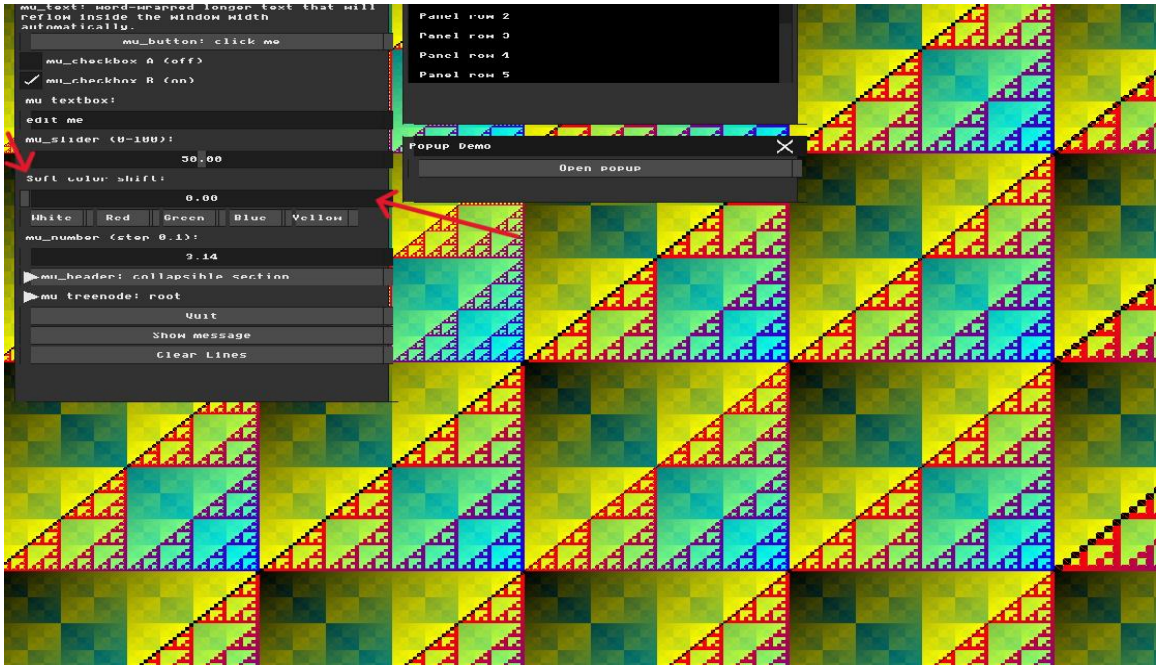
To connect the UI with the rendering system, I used the value of color_shift in the framebuffer rendering loop when computing the RGB color channels:

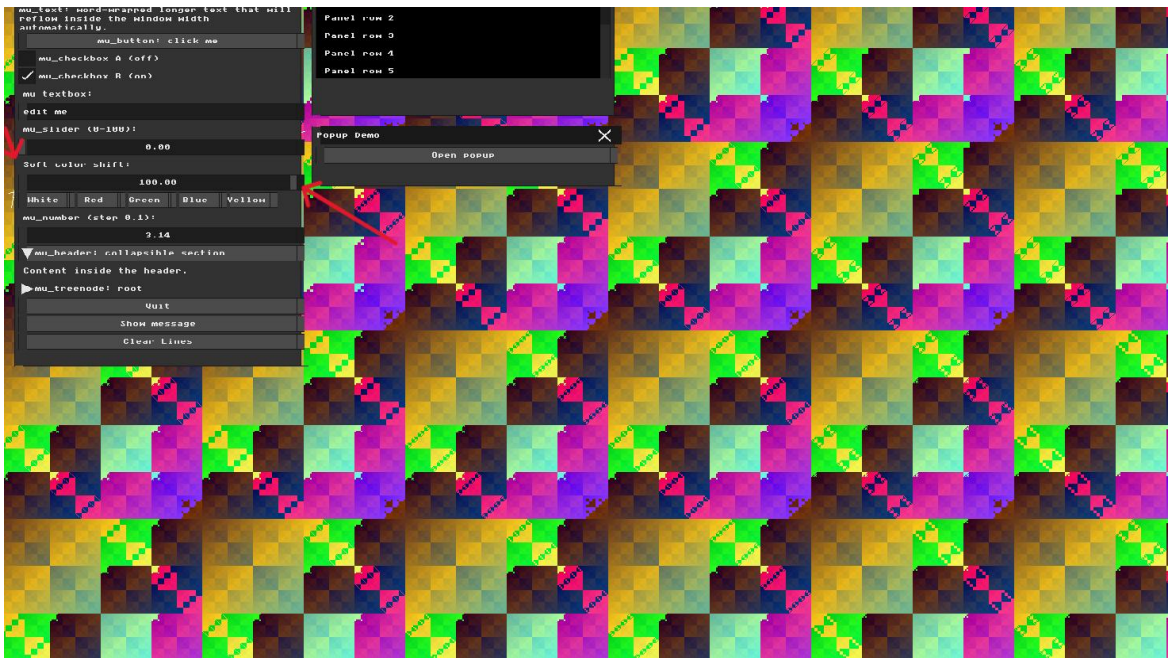
```
159           if (background_mode == 0)
160           {
161       r = ((x ^ y) + (int)color_shift) % 255;
162     g = ((x | y) + (int)(color_shift * 0.5f)) % 255;
163     b = ((x & y) + (int)(color_shift * 0.25f)) % 255;
164           }
```

Result

Moving the slider changes the colors of the generated background pattern in real time. The geometric structure of the pattern remains the same, while the color distribution gradually changes according to the slider value.

This demonstrates that the UI is successfully connected to the application state and that user interaction can directly influence the rendering output during execution.

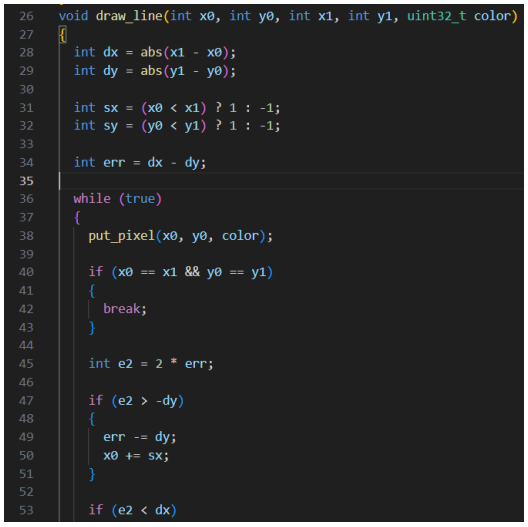




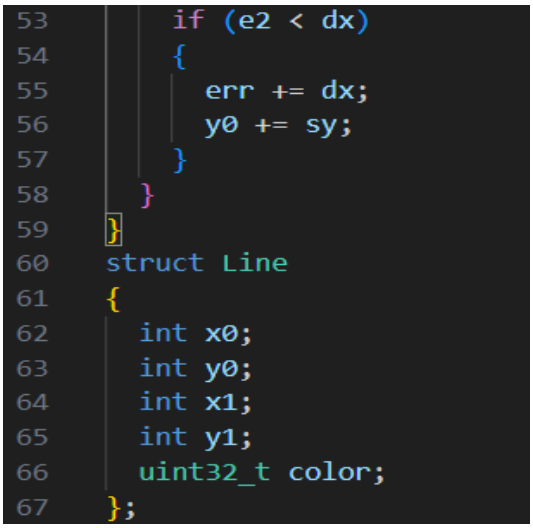
Part 6: Interactive Line Drawing App

The goal of this task was to implement Bresenham's Line Algorithm and integrate it with the existing Immediate Mode UI system to create an interactive drawing application.

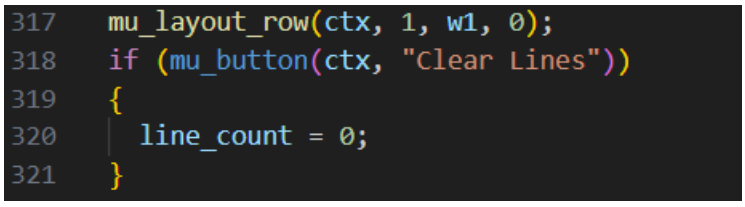
In main.cpp, I first implemented a put_pixel() function that writes a color value directly into the framebuffer. Using this function, I created a new draw_line() function based on Bresenham's algorithm. The implementation supports drawing lines in different directions and slopes while using only integer arithmetic. To make the application interactive, I added mouse input handling. The selected interaction method was click, drag, and release. When the user presses the left mouse button, the application stores the starting point of the line. While the mouse is being dragged, a temporary preview of the line is displayed. When the mouse button is released, the line is permanently stored and rendered on the screen.



```
26  void draw_line(int x0, int y0, int x1, int y1, uint32_t color)
27  {
28    int dx = abs(x1 - x0);
29    int dy = abs(y1 - y0);
30
31    int sx = (x0 < x1) ? 1 : -1;
32    int sy = (y0 < y1) ? 1 : -1;
33
34    int err = dx - dy;
35
36    while (true)
37    {
38      put_pixel(x0, y0, color);
39
40      if (x0 == x1 && y0 == y1)
41      {
42        break;
43      }
44
45      int e2 = 2 * err;
46
47      if (e2 > -dy)
48      {
49        err -= dy;
50        x0 += sx;
51      }
52
53      if (e2 < dx)
```



```
53        if (e2 < dx)
54        {
55          err += dx;
56          y0 += sy;
57        }
58      }
59    }
60    struct Line
61    {
62      int x0;
63      int y0;
64      int x1;
65      int y1;
66      uint32_t color;
67    };
```



```
317    mu_layout_row(ctx, 1, w1, 0);
318    if (mu_button(ctx, "Clear Lines"))
319    {
320      line_count = 0;
321    }
```

To support multiple drawings, I created a Line structure and an array that stores all previously drawn lines. Each frame, the application redraws all stored lines, allowing the user to create a persistent canvas.

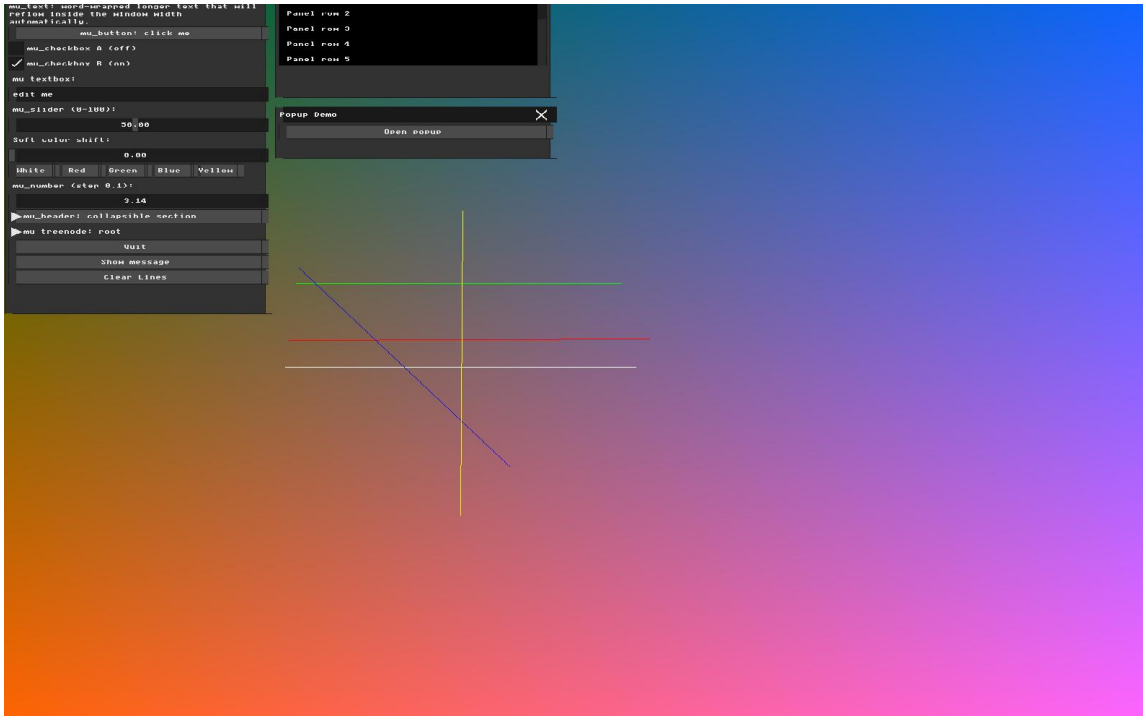
User Interface Extensions

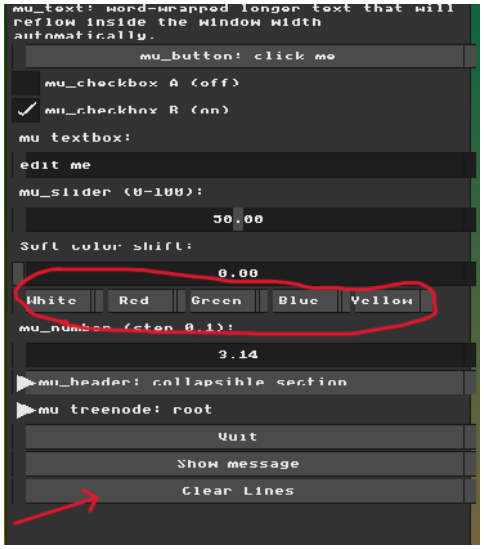
Several controls were added to improve usability:

- Color selection buttons (White, Red, Green, Blue, Yellow).
- A Clear Lines button that removes all stored lines.
- Real-time line preview while dragging the mouse.

Result

The final application allows the user to interactively draw multiple colored lines on the framebuffer. The implementation combines framebuffer manipulation, Bresenham line rasterization, mouse interaction, state management, and Immediate Mode UI concepts into a single drawing tool.




mu_text: word-wrapped longer text that will reflow inside the window width automatically.
mu_button: click me
mu_checkbox A (off)
mu_checkbox B (on)
mu textbox:
edit me
mu_slider (0-100):
50.00
Soft color shift:
0.00
White
Red
Green
Blue
Yellow
3.14
mu_header: collapsible section
mu treenode: root
Quit
Show message
Clear Lines